

GL-DASH

Sistema de Telemetria e Painel Digital Automotivo

VW Gol GL 1990 • Motor AP • Arduino Nano + OLED 2.42"

Documentação Técnica v2.0 • Revisada e Corrigida

Índice

1. Visão Geral do Projeto
2. Arquitetura de Hardware
3. Arquitetura de Software
4. Pinagem e Roteamento
5. Correções Críticas (v2.0)
6. Medidas de Segurança Elétrica
7. Guia de Montagem — Passo a Passo
8. Calibração e Testes

1. Visão Geral do Projeto

O GL-Dash é um sistema embarcado de telemetria e exibição de dados automotivos, desenvolvido para operar no ecossistema elétrico de um motor AP instalado em um VW Gol GL 1990. O projeto unifica três funções em uma única interface gráfica monocromática:

- **Tacômetro Digital (Conta-giros):** leitura em tempo real dos pulsos da bobina de ignição via interrupção de hardware.
- **Indicador Magnético de Marchas:** mapeamento da posição do trambulador via 4 sensores de Efeito Hall AH3503.
- **Telemetria Dinâmica (Força G):** medição de aceleração longitudinal via MPU-6050.

O design prioriza a instalação não-invasiva (padrão placa preta), sem cortes ou furos no painel original. Toda fixação é por dupla-face e conectores reversíveis.

i NOTA

Nome do projeto: GL-Dash. Mistura a versão clássica GL com a tecnologia digital. Alternativas consideradas: Telemetria AP, RetroDash 90, ShiftG-Tron.

2. Arquitetura de Hardware

O hardware é fisicamente dividido em três módulos para garantir integridade dos sinais e proteção contra interferências eletromagnéticas (EMI) da ignição.

Módulo 1 — Central Elétrica e Processamento

Localização: abaixo do painel, enclausurado em caixa de isolamento (Patola ou PVC).

- Microcontrolador: Arduino Nano (ATmega328P, 16 MHz, 2 KB RAM, 32 KB Flash)
- Condicionamento de energia: Step-Down LM2596, entrada 12V–14.4V, saída 5.0V estabilizados
- Filtros: capacitor 470 μ F (entrada) + capacitor 100 nF (saída do regulador)
- Isolamento galvânico RPM: optoacoplador PC817 + resistor 10 k Ω 2W + diodo 1N4007 + resistor pull-up 10 k Ω na saída
- Proteção de entrada: diodo 1N5408 em série na linha 12V (proteção contra inversão de polaridade)
- Proteção de sobrecarga: fusível automotivo 2A na linha pós-chave

⚠ CRÍTICO

CORREÇÃO v2.0: O resistor pull-up de 10 k Ω no pino 4 do PC817 (saída / D2) é obrigatório. Sem ele, o pino D2 flutua quando não há pulso da bobina, gerando RPM fantasma e interrupções espúrias.

Módulo 2 — Interface Gráfica / Cockpit

Localização: suporte (pod) impresso em 3D fixado na coluna de direção com dupla-face.

- Display: OLED 2.42" monocromático, interface I2C, chip SSD1309, endereço 0x3C
- Acelerômetro: MPU-6050, interface I2C, endereço 0x68 (pino AD0 fixado em GND)
- Desacoplamento local: capacitor 100 nF em cada módulo, soldado nos pinos VCC/GND

⚠ CRÍTICO

CORREÇÃO v2.0: O pino AD0 do MPU-6050 DEVE ser soldado ao GND para fixar o endereço I2C em 0x68. Se ficar flutuando, o endereço é imprevisível sob vibração e o sensor pode desaparecer do barramento.

Módulo 3 — Sensoriamento do Câmbio

Localização: PCB customizada (Projeto UPIR) abraçando a base da alavanca de câmbio.

- 4x sensores lineares AH3503 alimentados em 5V
- Ímãs de neodímio fixados na haste móvel do trambulador
- Conexão ao Módulo 1 via cabo manga blindado 6x22 AWG, malha aterrada apenas na Central

3. Arquitetura de Software

Leitura de RPM — Interrupção de Hardware

O pino D2 está configurado com `attachInterrupt()` no modo `FALLING`. Cada pulso do optoacoplador (originado da faísca da bobina) aciona a ISR instantaneamente, calculando o intervalo em microssegundos entre dois pulsos consecutivos.

```
volatile unsigned long lastPulse = 0;
volatile unsigned long pulseDelta = 0;
void IRAM_ATTR isr_rpm() {
    unsigned long now = micros();
    pulseDelta = now - lastPulse;
    lastPulse = now;
}
// No setup():
attachInterrupt(digitalPinToInterrupt(2), isr_rpm, FALLING);
```

! NOTA

O motor AP 4 cilindros gera 2 faíscas por rotação completa. $RPM = (60.000.000 / pulseDelta) / 2$

Renderização Gráfica — Arrays PROGMEM

As coordenadas do leque de RPM são pré-calculadas em Python no computador e armazenadas como arrays `int8_t` na memória Flash (PROGMEM). O Arduino lê esses valores e renderiza trapézios preenchidos com dois `drawTriangle()` por linha. Zero trigonometria em tempo real.

```
const int8_t fanXl[FAN_LINES] PROGMEM = { ... };
// Cada linha = 2 triangulos preenchidos:
u8g2.drawTriangle(ax,ay, bx,by, dx,dy); // triangulo 1
u8g2.drawTriangle(bx,by, cx,cy, dx,dy); // triangulo 2
```

⚠ CRÍTICO

CRÍTICO: use o construtor `_1_` (page buffer = 128 bytes) e `firstPage()/nextPage()`. O construtor `_F_` exige 1024 bytes de RAM — com o Nano o sistema trava silenciosamente.

Lógica de Marchas — Sensores Hall

Quatro portas analógicas leem os sensores Hall. A posição da alavanca é determinada pela combinação de quais sensores estão ativados (valor alto) simultaneamente. As marchas 3ª e 4ª são detectadas por combinações duplas:

Marcha	HALL0 (A0)	HALL3 (A1)	HALL1 (A2)	HALL2 (A3)
Neutro	Médio	Médio	Médio	Médio
1ª	Alto	—	—	—
2ª	—	—	Alto	—
3ª	Alto	—	Alto	—
4ª	—	Alto	—	Alto (leve)
5ª	—	Alto	—	—
Ré	—	—	—	Alto

⚠ ATENÇÃO

ATENÇÃO: Os valores exatos de 'Alto' e 'Médio' variam com a distância dos ímãs. Sempre calibrar fisicamente via Serial Monitor antes de definir os limiares no código.

Força G — MPU-6050 (Método Primário)

O MPU-6050 mede aceleração real no eixo X do veículo independentemente da marcha engajada. É o método primário e mais preciso. O eixo X do sensor deve ser alinhado com o eixo longitudinal do veículo durante a montagem no pod.

```
// Leitura do eixo X do MPU-6050:
int16_t ax_raw = mpu.getAccelerationX();
float gForce = ax_raw / 16384.0f; // sensibilidade +/-2g = 16384 LSB/g
```

⚠ ATENÇÃO

CORREÇÃO v2.0: O método de cálculo de Força G por delta de RPM foi descartado. Ele falha em ponto morto, embreagem pressionada e trocas de marcha. O MPU-6050 é o único sensor de G do sistema.

Watchdog Timer

O WDT reinicia automaticamente o Arduino se o loop() travar por mais de 250 ms — proteção essencial contra surtos elétricos e interferência EMI em ambiente automotivo.

```
#include <avr/wdt.h>
void setup() {
    wdt_enable(WDTO_250MS);
}
void loop() {
    wdt_reset(); // DEVE ser chamado a cada ciclo
    // ... resto do código
}
```

⚠ ATENÇÃO

Sem o Watchdog Timer, qualquer travamento causado por ruído elétrico congela o display permanentemente. O sistema só volta com desligar o carro.

4. Pinagem e Roteamento

Origem do Sinal	Tipo	Pino Arduino	Componente
PCB Câmbio — HALL0 (1ª marcha)	Analógico	A0	Sensor Hall AH3503
PCB Câmbio — HALL3 (5ª marcha)	Analógico	A1	Sensor Hall AH3503
PCB Câmbio — HALL1 (2ª marcha)	Analógico	A2	Sensor Hall AH3503
PCB Câmbio — HALL2 (Ré)	Analógico	A3	Sensor Hall AH3503
Barramento I2C	I2C — SDA	A4	OLED SSD1309 + MPU-6050
Barramento I2C	I2C — SCL	A5	OLED SSD1309 + MPU-6050
Optoacoplador PC817 (saída)	Digital Interr.	D2	Bobina de ignição (via opto)
Alimentação 5V (regulada)	Energia	5V	Step-Down LM2596 OUT+
GND	Energia	GND	Step-Down LM2596 OUT-

i NOTA

Protótipos de função obrigatórios no C++: declare todas as funções antes do setup() para evitar o erro 'was not declared in this scope' no compilador.

5. Correções Críticas (v2.0)

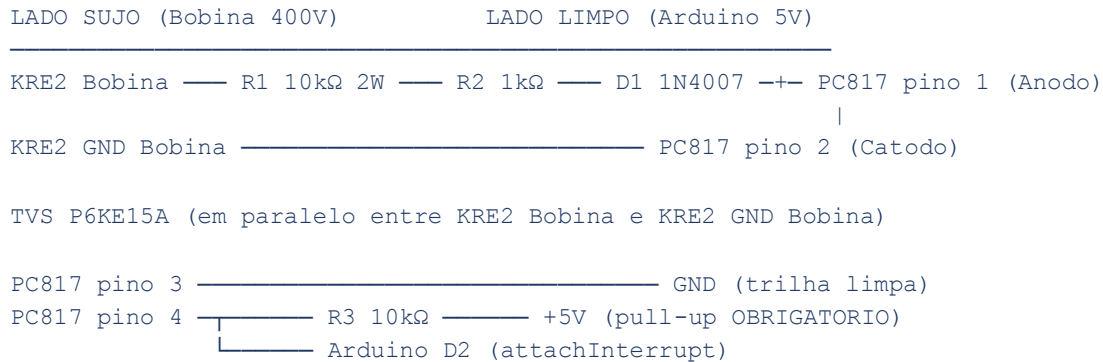
Esta seção detalha todos os problemas identificados na revisão técnica e suas soluções. Todas as correções estão incorporadas nos guias de montagem e no código gerado.

Prior.	Problema	Consequência	Solução
☐ CRÍTICO	Pull-up ausente no PC817	D2 flutua → RPM fantasma, sistema inutilizável	Adicionar R 10 kΩ entre pino 4 do PC817 e +5V
☐ CRÍTICO	Método de Força G indefinido	Dois sistemas conflitantes no código	MPU-6050 é o método oficial. Delta-RPM removido
☐ CRÍTICO	Construtor _F_ no U8g2	Nano fica sem RAM e trava silenciosamente	Usar construtor _1_ + firstPage()/nextPage()
☐ CRÍTICO	drawTriangle() mal usado	Leque não aparecia (linhas invisíveis)	Trapézio = 2× drawTriangle() (filled polygon)
☐ ALTO	TVS ausente na entrada da bobina	PC817 queima com spikes > 400V	Adicionar TVS P6KE15A em paralelo na entrada
☐ ALTO	Watchdog Timer ausente	Display congela sem recuperação automática	wdt_enable(WDTO_250MS) + wdt_reset() no loop
☐ ALTO	Protótipos C++ ausentes	Erro fatal: 'was not declared in this scope'	Declarar todos os protótipos antes do setup()
☐ MÉDIO	AD0 do MPU-6050 flutuando	Endereço I2C instável sob vibração	Soldar AD0 diretamente ao GND → endereço 0x68
☐ MÉDIO	Tabela de combinações Hall não documentada	Impossível recalibrar sem o autor	Tabela incluída na Seção 3 deste documento
☐ MÉDIO	Desacoplamento local ausente	Ruído I2C em acelerações bruscas	100 nF em cada módulo, nos pinos VCC/GND locais
☐ MÉDIO	Marcha inicial '3' no código	Estado incorreto na partida	Iniciar com 'N' (Neutro) é mais seguro
☐ BAIXO	Sem proteção contra inversão de polaridade	Risco em manutenção manual	Diodo 1N5408 em série na entrada 12V
☐ BAIXO	Resistor 10kΩ único antes do PC817	Corrente pode exceder o LED interno	Dois resistores: 10 kΩ 2W + 1 kΩ em série

6. Medidas de Segurança Eléctrica

Circuito Completo do Optoacoplador (RPM)

Este é o circuito mais crítico do projeto. O lado sujo (bobina) não deve cruzar eletricamente com o lado limpo (Arduino) em nenhuma hipótese.



⚠️ ATENÇÃO

O TVS P6KE15A deve ser inserido antes do R1, em paralelo com a entrada. Ele absorve spikes de até 600W em 1 ms, protegendo os resistores e o PC817 dos picos negativos da bobina.

Tabela de Componentes de Segurança

Componente	Valor / Modelo	Função	Localização
Fusível automotivo lâmina	2A	Proteção de sobrecarga na linha 12V	Em série na linha 15 (pós-chave)
Diodo de polaridade	1N5408 (3A)	Bloqueia inversão de polaridade	Em série antes do LM2596 IN+
TVS Transient Suppressor	P6KE15A	Absorve spikes da bobina > 15V	Paralelo na entrada do circuito opto
Resistor série 1 (opto)	10 kΩ 2W	Limita corrente / dissipa energia do pulso	Lado sujo, antes do R2
Resistor série 2 (opto)	1 kΩ	Protege o LED interno do PC817	Lado sujo, antes do PC817 pino 1
Diodo anti-reverso (opto)	1N4007	Bloqueia corrente reversa da bobina	Lado sujo, em série com PC817 pino 1
Pull-up saída PC817	10 kΩ	Mantém D2 em nível alto quando sem pulso	Lado limpo, entre PC817 pino 4 e +5V
Cap. filtro entrada	470 µF / 25V	Filtro de baixa frequência na entrada 12V	Próximo ao LM2596 IN+
Cap. filtro saída	100 nF	Filtro de alta frequência na saída 5V	Próximo ao LM2596 OUT+
Cap. desacoplamento OLED	100 nF	Filtra ruído local no módulo display	Soldado nos pinos VCC/GND do OLED
Cap. desacoplamento MPU	100 nF	Filtra ruído local no MPU-6050	Soldado nos pinos VCC/GND do MPU

7. Guia de Montagem — Passo a Passo

Siga as tabelas em ordem. Cada seção corresponde a um bloco funcional. Marque o checkbox ao concluir cada ligação. Nunca aplique energia antes de concluir a Etapa 0 (verificação visual).

✓ DICA

DICA DE SOLDAGEM: Use as pernas cortadas de resistores e capacitores para fazer pontes por baixo da placa de fenolite, unindo os pinos do soquete do Arduino até os bornes KRE. Cria trilhas rígidas superiores a pontinhos de estanho.

Etapa 0 — Lista de Materiais

Qtd.	Componente	Especificação	Módulo
1	Arduino Nano	ATmega328P, com soquete	Central
1	Step-Down LM2596	Módulo ajustável, ajustar para 5.0V antes de ligar	Central
1	Optoacoplador PC817	DIP-4	Central
1	Diodo 1N4007	1A / 1000V	Central (opto)
1	Diodo 1N5408	3A / 1000V	Central (entrada 12V)
1	TVS P6KE15A	Unidirecional, 15V	Central (opto)
2	Resistor 10 kΩ	1/4W — pull-up saída opto e divisor	Central
1	Resistor 10 kΩ	2W — série lado sujo opto	Central
1	Resistor 1 kΩ	1/4W — série lado sujo opto	Central
1	Capacitor 470 μF	25V eletrolítico — filtro entrada	Central
3	Capacitor 100 nF	50V cerâmico — filtros saída, OLED, MPU	Central + Cockpit
1	Fusível lâmina 2A	Com porta-fusível automotivo	Central
4	Borne KRE 2 pinos	Parafuso, passo 5mm — KRE 1 a 4	Central
1	Placa fenolite	Perfurada, 10×8 cm mínimo	Central
1	Caixa Patola / PVC	~12×8×4 cm	Central
1	OLED 2.42"	SSD1309, I2C, 128×64	Cockpit
1	MPU-6050	Módulo GY-521	Cockpit
4	Sensor Hall AH3503	Linear, 5V, montado na PCB UPIR	Câmbio
1	Cabo manga blindado	6 vias, 22 AWG, com malha	Câmbio → Central
4	Ímã de neodímio	N35 ou superior, disco 6×3 mm	Câmbio
1	Suporte (pod) 3D	PLA fosco preto/marrom, para coluna de direção	Cockpit

Etapa 1 — Usina de Energia e Filtros

Objetivo: criar a 'avenida de 5V' e a 'avenida GND' na placa perfurada. Ajuste o trimpot do LM2596 para exatamente 5.0V com multímetro ANTES de conectar qualquer componente.

✓	Origem	Destino	Função / Observação
☐	KRE1 → 12V Pós-Chave (Linha 15)	LM2596 → IN+	Energia bruta do carro
☐	KRE1 → GND Lateria (Linha 31)	LM2596 → IN-	Aterramento principal
☐	Diodo 1N5408 → Anodo	KRE1 → 12V (em série antes do LM2596 IN+)	Proteção anti-inversão de polaridade
☐	Capacitor 470 µF → Perna +	LM2596 → IN+ (paralelo)	Filtro de baixa frequência — entrada
☐	Capacitor 470 µF → Perna -	LM2596 → IN- (paralelo)	Filtro de entrada — GND
☐	LM2596 → OUT+ (5.0V)	Trilha 5V principal da placa	Cria a 'avenida 5V'
☐	LM2596 → OUT-	Trilha GND principal da placa	Cria a 'avenida GND'
☐	Capacitor 100 nF → Perna 1	Trilha 5V principal (paralelo)	Filtro fino de alta frequência — saída
☐	Capacitor 100 nF → Perna 2	Trilha GND principal (paralelo)	Filtro fino — GND
☐	Trilha 5V principal	Arduino Nano → Pino 5V (via soquete)	Alimenta o microcontrolador
☐	Trilha GND principal	Arduino Nano → Pino GND (via soquete)	Aterra o microcontrolador

⚠ CRÍTICO

Verifique com multímetro: deve ler 5.00V entre a trilha 5V e GND. Qualquer leitura acima de 5.2V pode danificar o Arduino e os sensores. Ajuste o trimpot do LM2596 antes de prosseguir.

Etapa 2 — Circuito de Proteção do RPM (PC817)

O lado sujo (pinos 1 e 2 do PC817) recebe a tensão da bobina. O lado limpo (pinos 3 e 4) pertence ao circuito do Arduino. Nunca conecte os dois lados à mesma trilha de GND.

✓	Origem	Destino	Função / Observação
☐	KRE2 → Sinal Bobina (+)	TVS P6KE15A → Perna 1 (paralelo com a entrada)	Absorve spikes transientes da bobina
☐	KRE2 → GND Bobina	TVS P6KE15A → Perna 2	GND do lado sujo (chassi/motor)
☐	KRE2 → Sinal Bobina (+)	Resistor 10 kΩ 2W → Perna 1	Entrada do circuito de proteção
☐	Resistor 10 kΩ 2W → Perna 2	Resistor 1 kΩ → Perna 1	Segunda etapa de limitação de corrente
☐	Resistor 1 kΩ → Perna 2	Diodo 1N4007 → Perna sem faixa (Anodo)	Bloqueia corrente reversa
☐	Diodo 1N4007 → Perna com faixa (Catodo)	PC817 → Pino 1 (Anodo LED)	Aciona o LED interno do opto
☐	KRE2 → GND Bobina	PC817 → Pino 2 (Catodo LED)	Fecha o circuito sujo
☐	PC817 → Pino 3 (Coletor)	Trilha GND principal (lado limpo)	Aterra o lado limpo do transistor
☐	Resistor 10 kΩ → Perna 1	Trilha 5V principal	Pull-up OBRIGATÓRIO na saída
☐	Resistor 10 kΩ → Perna 2	PC817 → Pino 4 (Emissor) E Arduino D2	Nó de saída: pull-up + sinal D2

⚠ CRÍTICO

O nó de junção entre o resistor pull-up 10kΩ, o pino 4 do PC817 e o pino D2 do Arduino é o ponto mais crítico do circuito. Sem o pull-up, D2 flutua e o Arduino conta giros fantasma mesmo com o motor desligado.

Etapa 3 — Sensoriamento do Câmbio (Trambulador)

A fiação desce pelo túnel do câmbio através do cabo manga blindado. A malha metálica externa é aterrada SOMENTE na ponta da Central Elétrica, nunca no câmbio.

✓	Origem	Destino	Função / Observação
☐	Trilha 5V principal	KRE3 → Pino 5V	Alimenta a PCB customizada dos Hall
☐	Trilha GND principal	KRE3 → Pino GND	Aterra a PCB do câmbio
☐	Arduino Nano → Pino A0	KRE3 → Sinal HALL0	1ª marcha — cabo verde (sugestão)
☐	Arduino Nano → Pino A1	KRE3 → Sinal HALL3	5ª marcha — cabo azul (sugestão)
☐	Arduino Nano → Pino A2	KRE3 → Sinal HALL1	2ª marcha — cabo amarelo (sugestão)
☐	Arduino Nano → Pino A3	KRE3 → Sinal HALL2	Marcha à Ré — cabo branco (sugestão)
☐	Malha blindada do cabo	Trilha GND principal (somente na Central)	Gaiola de Faraday — aterrar apenas aqui

i NOTA

3ª e 4ª marchas são detectadas por combinações duplas de sensores (ver Seção 3 — Tabela de Marchas). Não há fio dedicado para essas marchas: elas são lógica de software.

Etapa 4 — Cockpit (Display e Acelerômetro)

Chicote de 4 vias subindo da Central até o pod na coluna de direção. Instalar o capacitor de desacoplamento de 100 nF diretamente nos pinos VCC/GND de cada módulo, não na placa principal.

✓	Origem	Destino	Função / Observação
☐	Trilha 5V principal	KRE4 → Pino 5V	Alimenta OLED + MPU-6050
☐	Trilha GND principal	KRE4 → Pino GND	Aterramento do cockpit
☐	Arduino Nano → Pino A4	KRE4 → Sinal SDA	Barramento I2C — dados (verde)
☐	Arduino Nano → Pino A5	KRE4 → Sinal SCL	Barramento I2C — clock (amarelo)
☐	KRE4 → 5V (no cockpit)	OLED VCC + MPU-6050 VCC	Alimentação dos módulos
☐	KRE4 → GND (no cockpit)	OLED GND + MPU-6050 GND	GND dos módulos
☐	KRE4 → SDA (no cockpit)	OLED SDA + MPU-6050 SDA	I2C compartilhado em paralelo
☐	KRE4 → SCL (no cockpit)	OLED SCL + MPU-6050 SCL	I2C compartilhado em paralelo
☐	MPU-6050 → Pino AD0	GND (local no cockpit)	OBRIGATORIO — fixa endereço em 0x68
☐	Capacitor 100 nF	OLED VCC ↔ GND (soldado no módulo)	Desacoplamento local do display
☐	Capacitor 100 nF	MPU VCC ↔ GND (soldado no módulo)	Desacoplamento local do acelerômetro

⚠ ATENÇÃO

O MPU-6050 deve ser montado com o eixo X alinhado ao eixo longitudinal do veículo (frente-traseira). A aceleração de frenagem e arranque será lida corretamente apenas se a orientação estiver correta.

8. Calibração e Testes

Sequência de Testes — Em Bancada

1. Ajustar o trimpot do LM2596 para 5.00V com multímetro (sem nada conectado na saída).
2. Conectar apenas o Arduino Nano na trilha 5V. Verificar que o LED de power acende.
3. Carregar o sketch de teste de I2C (Scanner) e verificar que os endereços 0x3C (OLED) e 0x68 (MPU) aparecem no Serial Monitor.
4. Com o motor desligado, verificar que o Serial Monitor mostra RPM = 0 e nenhum pulso espúrio em D2.
5. Testar o circuito do PC817 com uma fonte de pulsos (gerador de função 12V, 50Hz). O Arduino deve contar pulsos corretamente.
6. Mover a alavanca de câmbio por todas as posições e registrar os valores raw de A0–A3 no Serial Monitor. Anotar os limiares.

Calibração dos Sensores Hall

No Arduino, carregar este sketch de calibração e anotar os valores para cada posição do câmbio:

```
void loop() {
  Serial.print("H0:"); Serial.print(analogRead(A0));
  Serial.print(" H3:"); Serial.print(analogRead(A1));
  Serial.print(" H1:"); Serial.print(analogRead(A2));
  Serial.print(" H2:"); Serial.println(analogRead(A3));
  delay(200);
}
```

Com os valores em mãos, preencher a tabela abaixo e depois ajustar os limiares no código:

Marcha	H0 (A0) raw	H3 (A1) raw	H1 (A2) raw	H2 (A3) raw
Neutro	_____	_____	_____	_____
1ª	_____	_____	_____	_____
2ª	_____	_____	_____	_____
3ª	_____	_____	_____	_____
4ª	_____	_____	_____	_____
5ª	_____	_____	_____	_____
Ré	_____	_____	_____	_____

Verificação Final — No Veículo

1. Com o carro desligado: verificar que o display mostra marcha 'N' e RPM 0.
2. Ligar o motor: verificar que o RPM sobe imediatamente na partida (800–1000 RPM em marcha lenta).
3. Com o motor em marcha lenta, engajar 1ª, 2ª, 3ª, 4ª, 5ª e Ré, verificando a troca no display.
4. Acelerar bruscamente e verificar a leitura de Força G positiva (tração) no display.
5. Frear bruscamente e verificar a leitura de Força G negativa (frenagem).
6. Verificar que ao desligar e religar a chave, o sistema inicializa corretamente (WDT ativo).

✓ DICA

Este documento é um registro vivo do projeto GL-Dash. Atualize os valores de calibração dos sensores Hall na tabela acima após a montagem final. Isso garante que qualquer manutenção futura possa ser feita sem depender de memória.

